Assignment Report

SFWRENG 3S03 (2026W)

Assignment 3

Author(s)

Salma Baig baigs19@mcmaster.ca baigs19

GitHub URL

`https://github.com/hydrowoxy/3S03-A3`

April 7, 2026

Original template credits: Samuel Zurowski and Charles Barone. Adapted for McMaster SFWRENG by Dr. Istvan David.

1 Specifying a Test Plan

Following is a test plan for the `Velocity` class. It covers the intended behaviour of the class, the need for review, and testing objectives at the specification, implementation, and interaction levels.

Plan Item	**Description**
Objectives for the class	The `Velocity` class must represent the motion of the puck using a non-negative scalar speed, a direction (in degrees), and corresponding horizontal and vertical components. These values must remain internally consistent at all times. Specifically, the class should correctly construct a velocity from a given speed and direction, maintain consistency between `speed`, `speedX`, and `speedY`, and ensure that the direction determines the correct sign of each component (negative when moving left or down). In addition, bounce operations such as `reverse`, `reverseX`, and `reverseY` must update the velocity in a way that reflects expected physical behaviour. Getter methods must always return the current valid state.
Inspection / review requirements	A design and code review is required prior to testing for several reasons: 1. Specification details are unclear. The informal description does not fully define how certain cases should be handled, such as direction values outside the range 0-359, the behaviour of the default constructor, or whether angles like 360° are equivalent to 0°. These ambiguities must be resolved before writing tests. 2. We want to ensure trigonometric and numerical correctness. The calculation of `speedX` and `speedY` depends on sine and cosine functions followed by conversion to integers. This introduces potential rounding and sign errors, especially at boundary angles such as 0°, 90°, 180°, and 270°. 3. We want to ensure the preservation of internal relationships. Any method that updates the velocity must maintain consistency between the scalar speed and its components. A review helps ensure that this relationship is re-established after every update. Without this review, test cases may reflect incorrect assumptions rather than actual intended behaviour.

Plan Item	Description
Specification-based testing objectives	When treating the class as a black box, the goal is to verify that each public method behaves according to its expected contract. Testing should include: • Typical cases. Representative speeds and directions across all four quadrants. • Boundary values. Directions such as 0, 90, 180, 270, and 359, and speeds such as 0 and large values. • Equivalence classes. Combinations of positive and negative values for `speedX` and `speedY`. • Invalid inputs (if applicable). For example, negative speeds or out-of-range directions. Tests should confirm that outputs match expectations and that the observable behaviour of the class aligns with its specification.
Implementation-based testing objectives	When treating the class as a white box, the goal is to exercise the internal logic and achieve strong structural coverage. This includes: • Statement and branch coverage, ensuring all methods and decision points are executed. • Condition coverage, testing how sign decisions are made based on direction. • Path coverage, particularly for direction normalization logic. • Rounding behaviour, verifying correctness near angles where integer rounding affects results (45°, 135°, 225°, 315°). These tests increase confidence that the implementation correctly handles all internal cases.

Plan Item	Description
Interaction-based testing objectives	The goal is to verify that sequences of method calls produce consistent and expected results, and that operations compose correctly. Key interaction checks include: • `setDirection` updates components without changing speed. • `setSpeed` rescales components while preserving direction. • Applying `reverseX` and `reverseY` together produces the same result as `reverse`. • Applying reverse operations twice returns the object to its original state. • Any sequence of updates followed by getters yields a state that remains internally consistent. These tests ensure that method interactions do not violate expected behaviour over time.

2 Contract-Based Specification

Boolean expressions are defined in pseudocode. The notation `old(e)` refers to the value of an expression before execution. Directions are assumed to be in degrees and within the range $[0, 360)$.

Class Invariant

```
invariant:
    speed >= 0
    && 0 <= direction < 360
    && speedX == round(speed * cos(direction))
    && speedY == round(speed * sin(direction))
    && abs(speedX) <= speed
    && abs(speedY) <= speed
    // sign constraints based on direction
    && (direction > 90 && direction < 270 ==> speedX <= 0)
    && (direction < 90 || direction > 270 ==> speedX >= 0)
    && (direction > 180 && direction < 360 ==> speedY <= 0)
    && (direction > 0 && direction < 180 ==> speedY >= 0)
```

Constructors

Velocity()

Pre: true

Post:

```
speed == 0
&& direction == 0
&& speedX == 0
&& speedY == 0
&& invariant holds
```

Velocity(Speed speed, Direction direction)

Pre:

```
speed != null && direction != null
&& speed >= 0
&& 0 <= direction < 360
```

Post:

```
this.speed == speed
&& this.direction == direction
&& this.speedX == round(speed * cos(direction))
&& this.speedY == round(speed * sin(direction))
&& invariant holds
```

Getters

getSpeed()

Pre: true

Post:

```
result == this.speed
&& result >= 0
&& object state remains unchanged
```

getSpeedX()

Pre: true

Post:

```
result == this.speedX
&& abs(result) <= this.speed
&& object state remains unchanged
```

getSpeedY()

Pre: true

Post:

```
result == this.speedY
```

```
&& abs(result) <= this.speed
&& object state remains unchanged
```

getDirection()

Pre: true

Post:

```
result == this.direction
&& 0 <= result < 360
&& object state remains unchanged
```

Setters

setSpeed(Speed speed)

Pre:

```
speed != null && speed >= 0
```

Post:

```
this.speed == speed
&& this.direction == old(this.direction)
&& this.speedX == round(speed * cos(this.direction))
&& this.speedY == round(speed * sin(this.direction))
&& invariant holds
```

setDirection(Direction direction)

Pre:

```
direction != null && 0 <= direction < 360
```

Post:

```
this.direction == direction
&& this.speed == old(this.speed)
&& this.speedX == round(this.speed * cos(direction))
&& this.speedY == round(this.speed * sin(direction))
&& invariant holds
```

Direction Reversal Methods

reverse()

Pre: true

Post:

```
this.speed == old(this.speed)
&& this.direction == (old(this.direction) + 180) mod 360
```

```
&& this.speedX == -old(this.speedX)
&& this.speedY == -old(this.speedY)
&& invariant holds
```

reverseX()

Pre: true

Post:

```
this.speed == old(this.speed)
&& this.speedX == -old(this.speedX)
&& this.speedY == old(this.speedY)
&& this.direction == (540 - old(this.direction)) mod 360
&& invariant holds
```

reverseY()

Pre: true

Post:

```
this.speed == old(this.speed)
&& this.speedY == -old(this.speedY)
&& this.speedX == old(this.speedX)
&& this.direction == (360 - old(this.direction)) mod 360
&& invariant holds
```

3 Test Cases

Each test case specifies the input state, the operation performed, and the expected outcome.

setDirection

Test 1: updating direction preserves speed and updates components

Input: Initial state: v = new Velocity$(10, 0)$ Call: v.setDirection(90)

Expected Output:

```
v.getDirection() == 90
v.getSpeed() == 10
v.getSpeedX() == 0
v.getSpeedY() == 10
class invariant holds
```

setSpeed

Test 1: updating speed preserves direction and rescales components

Input: Initial state: v = new Velocity$(5, 180)$ Call: v.setSpeed(20)

Expected Output:

```
v.getSpeed() == 20
v.getDirection() == 180
v.getSpeedX() == -20
v.getSpeedY() == 0
class invariant holds
```

reverse

Test 1: reversing motion from right to left

Input: v = new Velocity$(10, 0)$ Call: v.reverse$()$

Expected Output:

```
direction == 180
speedX == -10
speedY == 0
speed == 10
```

Test 2: reversing from northeast to southwest

Input: v = new Velocity$(14, 45)$ Call: v.reverse$()$

Expected Output:

```
direction == 225
speedX == -10
speedY == -10
speed == 14
```

Test 3: applying reverse twice restores original state

Input: v = new Velocity$(8, 123)$ Call: v.reverse$()$ twice

Expected Output:

```
Final (speed, direction, speedX, speedY)
matches the original state
```

Test 4: reversing straight downward motion

Input: v = new Velocity$(7, 270)$ Call: v.reverse$()$

Expected Output:

```
direction == 90
speedX == 0
speedY == 7
speed == 7
```

Test 5: reversing when speed is zero

Input: v = new Velocity$(0, 60)$ Call: v.reverse()

Expected Output:

```
direction == 240
speedX == 0
speedY == 0
speed == 0
```

reverseX

Test 1: bounce off vertical wall while moving right

Input: v = new Velocity$(10, 0)$ Call: v.reverseX()

Expected Output:

```
speedX == -10
speedY == 0
direction == 180
speed == 10
```

Test 2: northeast becomes northwest

Input: v_1 = new Velocity$(14, 45)$ v_2 = new Velocity$(14, 45)$ Call: v_1.reverseX()

Expected Output:

```
v_1.direction == 135
v_1.speedX == -v_2.speedX
v_1.speedY == v_2.speedY
speed == 14
```

Test 3: vertical motion remains unchanged

Input: v = new Velocity$(10, 90)$ Call: v.reverseX()

Expected Output:

```
speedX == 0
speedY == 10
direction == 90
speed == 10
```

Test 4: applying reverseX twice restores original state

Input: v = new Velocity$(12, 200)$ Call: v.reverseX() twice

Expected Output:

```
Final state identical to initial state
(speed, direction, speedX, speedY)
```

Test 5: reverseX followed by reverseY equals reverse

Input: a = new Velocity$(10, 30)$ b = new Velocity$(10, 30)$

Call: a.reverseX(); a.reverseY(); b.reverse();

Expected Output:

```
a.speed == b.speed
a.direction == b.direction
a.speedX == b.speedX
a.speedY == b.speedY
```

4 Test Automation